

Стъпка 1 Обобщение — Основи на обучението с подсилване

Изследване върху възможностите за прилагане на изкуствен интелект в компютърните игри

Alexander Andreev

April 2026

Съдържание

1	Стъпка 1 — Основи на обучението с подкрепление	1
1.1	Как обучението с подкрепление се утвърди като самостоятелна дисциплина	1
1.2	Основни принципи — агент, действие, среда	2
1.3	Формулировка на MDP	3
1.4	Информационни множества — когато не можете да видите всичко	4
1.5	Динамично програмиране — При наличие на модел	5
1.6	Обучение с темпорална разлика — Обучение без модел	6
1.7	DQN — Дълбоки Q-мрежи	7
1.8	PPO — Оптимизация на проксималната стратегия	8
1.9	Практическо валидиране — собствени имплементации спрямо stable-Baselines3	9

1 Стъпка 1 — Основи на обучението с подкрепление

Това е съкратен обзор на основните материали по обучение с подкрепление, разгледани в Стъпка 1. Той е предназначен за две цели: да служи като бърза справка при преминаване към следващите стъпки и като основен източник за **синтеза** на публичния доклад от Стъпка 15.

1.1 Как обучението с подкрепление се утвърди като самостоятелна дисциплина

Обучението с подкрепление не възниква изведнъж като завършена дисциплина. Неговите корени се простират в множество области, които дълги десетилетия почти не са били свързани помежду си.

Най-ранната нишка идва от **животинската психология** в началото на XX век — „Законът за ефекта“ на Торндайк (1911) предполага, че действията, последвани от удовлетворяващи резултати, стават по-вероятни. Това всъщност представлява идеята за **сигнал за награда**. Успоредна нишка преминава през **теорията на оптималното управление** през 50-те години на миналия век, където Белман разработва **динамично програмиране** за решаване на **последователни проблеми на вземане на решение**. Той въвежда **функцията на стойността** и **принципа на оптималност**, които и двете са в **ядрото** на съвременното обучение с подкрепление. Трета нишка идва от **обучението чрез проба и грешка** в ранните изследвания в областта на

изкуствения интелект — програмата за дама на Самюел (1959) се учи, като играе срещу себе си, коригирайки своята **оценъчна функция** въз основа на победи и загуби.

Тези нишки остават отделни до 80-те и 90-те години на миналия век. Докторската работа на Сътън върху обучението с темпорална разлика (1988) запълва празнината между теориите за животинското учене и динамичното програмиране на Белман. След това Уоткинс формализира q-обучението (1989), предоставяйки на областта първия ясен, безмоделен алгоритъм за управление със сходимостни гаранции. Към момента, когато Сътън и Барто публикуват своя учебник през 1998 г. (актуализиран през 2018 г.), обучението с подкрепление вече се е утвърдило като разпознаваема дисциплина със собствена нотация, таксономия на проблемите и алгоритмичен инструментариум.

Революцията в дълбокото обучение достига до обучението с подкрепление през 2013–2015 г., когато DQN на DeepMind се научава да играе Atari Games, използвайки само сурови пиксели. Тази демонстрация ясно показва, че алгоритмите за обучение с подкрепление, комбинирани с невронни апроксиматори на функции, могат да решават високоизмерни проблеми от реалния свят. PPO се появява през 2017 г. като практичен метод на градиента на стратегията и оттогава обучението с подкрепление се разширява в областите на роботиката, игровия изкуствен интелект, системите за препоръки и привеждането в съответствие на езикови модели (RLHF).

Важното е да се подчертае, че обучението с подкрепление не е просто „машинно обучение с награди“. То представлява самостоятелна рамка за последователно вземане на решения при несигурност, със собствени теоретични основи, произтичащи от теорията на управлението, психологията и статистиката.

Прочетете повече: Sutton, R.S. & Barto, A.G. (2018). *Обучение с подкрепление: Въведение*, второ издание — Глава 1.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.2 Основни принципи — агент, действие, среда

Цялата рамка на обучението с подкрепление се основава на прост цикъл. **Агентът** наблюдава текущото **състояние** на **средата**, избира **действие** и получава сигнал за **награда** заедно с ново **състояние**. След това процесът се повтаря. Целта на агента е да научи **стратегия** — съпоставяне между състояния и действия — която максимизира натрупаната награда във времето.

Няколко уточнения относно терминологията:

- **Състояние** (s): описание на средата в даден момент. В cartPole това включва позицията на количката, скоростта на количката, ъгъла на пръта и ъгловата скорост на пръта. В покер биха били раздадените карти и историята на залозите.
- **Действие** (a): възможните действия на агента. Те могат да бъдат дискретни (например бутане наляво или надясно) или непрекъснати (например прилагане на конкретен въртящ момент).
- **Награда** (r): скаларна величина за обратна връзка, която агентът получава след всяко действие. Наградите могат да бъдат оскъдни (само в края на играта) или гъсти (на всеки времеви интервал).

- **Стратегия (π):** стратегията на агента. Тя може да бъде детерминистична ($\pi(s) = a$) или стохастична, като $\pi(a|s)$ дава разпределение на вероятностите върху възможните действия.
- **Функция на стойността ($V^\pi(s)$):** очакваната натрупана награда от състояние s нататък, ако агентът следва стратегия π . Тя показва на агента колко добро е едно състояние в дългосрочен план, а не само непосредствено.
- **Коефициент на дисконтиране (γ):** число между 0 и 1, което определя доколко агентът отчита бъдещите награди спрямо непосредствените такива. Висока стойност на $\gamma = 0.99$ означава, че агентът е търпелив; ниска стойност на $\gamma = 0.5$ показва, че е късоглед.

Критичното предизвикателство при проектирането на системи за обучение с подкрепление е **компромисът изследване–експлоатация**. Агентът трябва да изследва непознати действия, за да открие потенциално по-добри стратегии, но същевременно и да експлоатира наличните си знания, за да събира награда. Прекомерното изследване води до загуба на време, а прекалената експлоатация може да доведе до заседналост в локален оптимум.

Съществуват два основни класа алгоритми. **Методите, базирани на стойност** (като DQN) научават стойността на състоянията или двойките състояние-действие и извеждат стратегия въз основа на тези стойности. **Методите, базирани на политика** (като PPO) научават политиката директно. И двата подхода имат своите предимства; значителна част от областта на обучението с подкрепление е посветена на тяхното ефективно комбиниране.

Прочетете повече: OpenAI Spinning Up — „Ключови концепции в обучението с подкрепление“
https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

1.3 Формулировка на MDP

Марковски процес на вземане на решения (MDP) представлява формалната математическа рамка, върху която са изградени повечето алгоритми за обучение с подкрепление. Той включва пет основни компонента:

- S : множество от състояния
- A : множество от действия
- $P(s'|s, a)$: функция на преход — вероятност за преминаване в състояние s' при дадено състояние s и действие a
- $R(s, a, s')$: функция на наградата
- $\gamma \in [0, 1]$: коефициент на дисконтиране

Частта „Марков“ означава, че бъдещето зависи единствено от текущото **състояние**, а не от историята на пътя до него. Това е силно предположение — и такова, което не е възникнало в завършен вид в много интересни ситуации (като **покер**, където историята на залозите има значение). Разглеждаме този въпрос в Раздел 4.

Целта е да се намери стратегия π^* , която максимизира очакваната дисконтирана възвръщаемост.

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Ключовата рекурсивна зависимост е **уравнението на Белман**:

$$V^{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^{\pi}(s')]$$

Това означава, че стойността на едно състояние е равна на очакваната непосредствена награда плюс дисконтираната стойност на следващото състояние. Цялото динамично програмиране, tD learning и дълбокото обучение с подсилване се основават на вариации на това уравнение.

Прочетете повече: Сътон и Барто (2018). *Обучение с подкрепление: Въведение*, глава 3 — Крайни марковски процеси на вземане на решения.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.4 Информационни множества — когато не можете да видите всичко

Стандартните MDP предполагат, че агентът може напълно да наблюдава състоянието. В действителност обаче често това не е така. В покера например не виждате картите на опонента, а в игрите със стратегия в реално време действа т.нар. **мъгла на войната**. Това са **частично наблюдаеми** среди.

Формалното разширение е **Partially Observable MDP (POMDP)**, който включва функция за наблюдение — агентът получава наблюдения, представляващи шумни или непълни изгледи на истинското състояние. Но POMDP са печално известни с това, че оптималното им решаване е изключително трудно.

В теорията на игрите същата идея се изразява чрез **информационни множества**. Информационно множество обединява всички състояния на играта, които играчът не може да различи едно от друго въз основа на направените наблюдения. В кун покер (3 карти, 2 играчи), когато държите поп и все още не е имало залагания, вие се намирате в информационно множество — знаете своята карта, но не и тази на опонента си, така че не можете да определите дали действителното състояние е „Аз имам поп, опонентът има дама“ или „Аз имам поп, опонентът има цар“.

Алгоритмите, които работят в тези среди — като CFR (минимизиране на контрафактичното съжаление, разгледано в стъпка 2) — оперират върху информационни множества, а не върху отделни състояния. Това представлява фундаментално различен изчислителен проблем от стандартното RL и е причината изкуственият интелект за игра на игри да се нуждае от собствен набор инструменти, различни от тези, които DQN или PPO предоставят по подразбиране.

Разбирането на информационните множества вече е от значение, защото следващите стъпки (5–8) се занимават изцяло с игри с **непълна информация**. Алгоритмите се променят, но **ядрото** на концепцията остава:

вие вземате решения въз основа на това, което можете да наблюдавате, като разсъждавате върху това, което не можете.

Прочетете още: Shoham, Y. & Leyton-Brown, K. (2008). *Многоагентни системи: Алгоритмични, игровитеоретични и логически основи*, Глава 5.

Безплатно: <http://www.masfoundations.org/download.html>

1.5 Динамично програмиране — При наличие на модел

Динамичното програмиране (DP) представлява най-ранният подход за решаване на MDP. То е приложимо, когато разполагате с пълна информация за средата: вероятностите за преход $P(s'|s, a)$ и функцията на наградата $R(s, a, s')$. На практика това означава, че правилата на играта са ви напълно известни.

Двата основни алгоритъма при динамичното програмиране са:

Итерация на политиката редува два етапа. Първо се извършва *оценка на стратегията*: при дадена фиксирана стратегия се изчислява стойността на всяко състояние чрез многократно прилагане на уравнението на Белман, докато не се постигне сходимост. След това следва *подобрене на стратегията*: стратегията се актуализира така, че да бъде алчна спрямо новоизчислените стойности. Процесът се повтаря, докато стратегията престане да се променя. Гарантирано е, че тя сходява към оптималната стратегия.

Итерация на стойностите е пряк път. Вместо да се оцени напълно една **стратегия**, преди тя да бъде подобрена, тя комбинира оценката и подобренieto в една единствена актуализация. На всяка стъпка, за всяко **състояние**, се избира **действието**, което максимизира очакваната едностъпкова възвръщаемост плюс дисконтираната стойност на следващото **състояние**. Тя сходява към оптималната функция на стойността, от която се извлича оптималната **стратегия**.

DP е полезен, защото осигурява точни решения и има ясни гаранции за сходимост. Недостатъкът е очевиден: необходим е пълният модел, а изчислението нараства с броя на състоянията. За кун покер с 12 състояния DP е тривиален. За Го със 10^{170} състояния това е невъзможно.

Причината DP да има значение, въпреки че рядко го използваме директно, е, че всеки следващ метод — метод Монте Карло, tD learning, DQN, PPO — може да се разглежда като приближение към решението на DP, когато моделът е неизвестен или пространството на състоянията е твърде голямо. DP представлява теоретичния таван, към който практическите алгоритми се стремят да се доближат.

Прочетете повече: Сътон и Барто (2018). *Обучение с подкрепление: Въведение*, Глава 4 — Динамично програмиране.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.6 Обучение с темпорална разлика — Обучение без модел

Обучението с темпорална разлика (TD) е пробивът, който направи обучението с подкрепление приложимо на практика. За разлика от динамичното програмиране, то не изисква модел на средата. За разлика от методите на Монте Карло, то не е необходимо да чака края на един епизод, за да се учи. То актуализира стойностните оценки след всяка стъпка, използвайки самоподкрепяне: текущата оценка на стойността на следващото състояние замества истинската бъдеща възвръщаемост.

Основната актуализация на TD(0) за функцията на стойността е:

$$V(s_t) \leftarrow V(s_t) + \alpha [r_t + \gamma V(s_{t+1}) - V(s_t)]$$

Терминът в скобите е **tD грешка** — разликата между настъпилото (награда плюс оценена бъдеща стойност) и предвиденото. Ако tD грешката е положителна, състоянието се е оказало по-добро от очакваното; ако е отрицателна, то е било по-лошо.

Контролната версия на тази идея е **q-обучение** (Watkins, 1989):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

q-обучението научава стойността на двойки състояние-действие и използва оператора $\max$, за да бъде **извънполитиково** — то се учи относно оптималната стратегия дори докато следва изследователска. Това го прави толкова практично.

tD learning е по-изгодно от динамичното програмиране, тъй като не изисква модел. То е по-изгодно и от метода Монте Карло, защото се учи онлайн (на всяка стъпка, а не на всеки **епизод**) и има по-ниска **дисперсия**. **Самоподкрепянето** въвежда известно **пристрастие**, но в практиката тази размяна е изключително благоприятна. tD методите са основата, върху която се градят DQN и повечето съвременни **алгоритми, базирани на стойности**.

SARSA е вариантът на q-обучението, който работи по текущата политика — той актуализира стойностите, използвайки действието, което агентът действително предприе след това, а не най-доброто възможно действие. Това го прави по-консервативен и понякога по-безопасен на практика, но не му позволява да научи оптимална стратегия, докато следва под-оптимална изследователска стратегия.

Прочетете повече: Сътон и Барто (2018). *Обучение с подкрепление: Въведение*, Глава 6 — Обучение с темпорална разлика.

Безплатно: <http://incompleteideas.net/book/the-book-2nd.html>

1.7 DQN — Дълбоки Q-мрежи

DQN (Mnih и др., 2015) е алгоритъмът, който доказва възможността дълбоките невронни мрежи да се използват като апроксиматори на функция в обучението с подкрепление в голям мащаб. Преди DQN обучението с Q-стойности беше ограничено до таблична среда или ръчно извлечени признаци. DQN показва, че може да се научи да играе 49 Atari Games директно от суров пикселов вход, като надмина човешкото представяне в много от тях.

Как работи. DQN заменя Q-таблицата с невронна мрежа, която приема състояние като вход и извежда q-стойности за всички действия. Обучението на мрежата се извършва чрез минимизиране на квадратичната TD грешка — разликата между предсказаната Q-стойност и целта (награда плюс отстъпена максимална Q-стойност на следващото състояние). На всяка стъпка агентът избира действието с най-висока q-стойност (експлоатация) или произволно действие с вероятност ϵ (изследване).

Два ключови похвата осигуряват достатъчна стабилност, за да работи този подход:

1. **Повторение на натрупан опит.** Вместо да се обучава върху последователни преходи (които са силно корелирани), преходите се съхраняват в голям буфер и се извличат на случаен принцип в минипартиди. Това нарушава корелацията, намалява дисперсията и позволява всеки опит да бъде използван многократно. Принципът е аналогичен на разбъркването на тренировъчните данни при обучение с учител.
2. **Целева мрежа.** Поддържа се отделно копие на q-мрежата, което се обновява само периодично (на всеки няколко хиляди стъпки или епизода). TD target се изчислява с помощта на това замразено копие. Без него целта се измества при всяка тренировъчна стъпка — вие преследвате подвижна цел, което води до колебания и разминаване. Целевата мрежа разделя процеса на обновяване и стабилизира обучението.

Защо DQN е важна в сравнение с предишните методи: табличното Q-обучение не може да се справи с високоизмерни състояния (изображения, непрекъснати наблюдения); апроксимацията на функция само с Q-обучение беше известна като нестабилна („смъртоносната троица“ от извънполитиково + апроксимация на функция + самоподкрепяне). Двата трика на DQN — повторение на натрупан опит и целеви мрежи — адресираха нестабилността, правейки дълбокото обучение с базирано на стойност подсилващо обучение практически за първи път.

Ограничения. DQN работи единствено с дискретни пространства на действията. Тя може да надценява q-стойности (решено от Double DQN). Като извънполитиков метод, тя е ефективна по отношение на пробите, но това може да доведе до остаряла информация в буфера за възпроизвеждане. И най-важното — тя научава алчна детерминистична стратегия, без естествен начин за изразяване на стохастични стратегии, което е от значение в игрите, където смесените стратегии са оптимални.

В нашата реализация от Стъпка 1 DQN реши CartPole-v1 (постигайки средна стойност за епизод от 477,5 спрямо цел от 475) след приблизително 1000 епизода обучение. Най-влиятелните хиперпараметри бяха размерът на мрежата, епсилон минимумът и честотата на синхронизация на целевата мрежа.

Бележка относно невронните мрежи в тази стъпка. q-мрежата в DQN е стандартен многослоен пер-

цептрон (MLP), използван тук като практичен инструмент — функция, която съпоставя състояния към q -стойности. За Стъпка 1 вътрешните механизми на невронната мрежа (инициализация на теглоите, потокът на градиента, функциите на активиране) бяха разглеждани като черна кутия: използвахме настройките по подразбиране на `pyTorch` и се фокусирахме върху самия RL алгоритъм. По-задълбочено разглеждане на архитектурите на невронните мрежи и тяхното взаимодействие с методите за намиране на равновесие ще бъде представено в Стъпка 5 (невронно приближение на равновесие).

Прочетете повече: Mnih, V. и др. (2015). „Контрол на човешко ниво чрез дълбоко обучение с подкрепление.“ *Nature*, 518(7540), 529–533.
arXiv: <https://arxiv.org/abs/1509.06461>

1.8 PPO — Оптимизация на проксималната стратегия

PPO (Schulman и др., 2017) прилага фундаментално различен подход към RL в сравнение с DQN. Вместо да изчислява стойността на действията и да извежда стратегия въз основа на тези стойности, PPO **научава стратегията директно** — невронна мрежа генерира разпределение на вероятностите върху действията, а мрежата се обучава да увеличава вероятността на действията, които водят до високи възнаграждения.

Проблемът, който PPO решава. Методите на градиента на стратегията са мощни в теорията, но са печално известни с нестабилността си на практика. Основният алгоритъм **REINFORCE** страда от висока дисперсия и може да предприема разрушително големи стъпки, които провалят добра стратегия. **Trust Region Policy Optimization** (TRPO, Schulman и др., 2015) се справи с това, като ограничи колко може да се промени стратегията при всяка актуализация, но TRPO изисква скъпа оптимизация от втори ред (изчисляване на хесиана).

Как работи PPO. PPO запазва идеята за стабилност от TRPO, но заменя ограничението с много по-прост механизъм: **отсичане**. Ключовото количество е отношението на вероятностите $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ — колко по-вероятно (или по-малко вероятно) е новата стратегия да предприеме същото действие, което старата стратегия е предприела. PPO отсича това отношение в диапазона $[1 - \epsilon, 1 + \epsilon]$ (обикновено $\epsilon = 0.2$), което означава:

- Ако едно **действие** се окаже добро (положително **предимство**) и отношението на вероятностите надхвърли $1 + \epsilon$, **градиентът** се анулира — **стратегията** не трябва да става прекалено „алчна“ за това **действие**.
- Ако едно **действие** се окаже лошо (отрицателно **предимство**) и отношението на вероятностите падне под $1 - \epsilon$, **градиентът** също се анулира — така се предотвратява прекомерна корекция на **стратегията**.

Това е **изрязаната сурогатна цел** и именно тя осигурява работата на PPO. Тя постига стабилност, сходна с тази на TRPO, като използва единствено градиенти от първи ред (без хесиан), което значително намалява необходимата изчислителна мощност и улеснява имплементацията ѝ.

Архитектура. PPO е актьор-критик: **мрежа на стратегията** (актьор) извежда вероятности за действия, а **мрежа за стойности** (оценител) оценява стойностите на състоянията. Мрежата за стойности позволява прилагането на обобщено изчисляване на предимството (GAE), което осигурява нискодисперсни оценки

на предимството, използвани при **градиента на стратегията**. И двете мрежи се обучават едновременно.

Защо PPO е важен в сравнение с DQN: PPO естествено работи както с дискретни, така и с непрекъснати пространства на действията. Той може да усвоява **стохастични стратегии**, което е от решаващо значение в игрови среди, където прилагането на **смесени стратегии** е необходимо. Тъй като е **политика на работа** (използва пресни данни от **текущата стратегия**), той избягва проблемите със застояли данни. А изрязаната цел го прави забележително устойчив — PPO е предпочитаният избор за повечето RL приложения днес, включително RLHF за езикови модели.

Ограничения. Като **политика на работа**, PPO е по-малко **ефективен по отношение на пробите** в сравнение с **извънполитикови** методи като DQN (всеки опит се използва само за няколко **епохи на обновяване**, преди да бъде отхвърлен). Той изисква внимателна настройка на оценката на **предимството (GAE Lambda)**, **бонуса за ентропия** и архитектурата на **мрежата**. Освен това все още може да среща трудности при оскъдни награди — ако **агентът** рядко постига успех, наличният сигнал за учене е недостатъчен.

В нашата реализация от Стъпка 1 PPO успя да реши lunarLander-v3 (средна стойност за епизод от 202,2 при цел от 200) след приблизително 264 хиляди стъпки на средата. Решаващи фактори бяха капацитетът на мрежата (удвоен от [64,64] на [128,128]) и нормализацията на предимството.

Бележка относно невронните мрежи в тази стъпка. Както и при DQN, мрежите за **стратегия** и **стойност** тук са стандартни MLPs, чиито вътрешни механизми не бяха във фокуса на тази стъпка. Приехме прагматичен подход „ако работи“ към невронните компоненти и се концентрирахме върху разбирането на логиката на алгоритъма PPO (**отсичане**, GAE, структура на **разгръщане**). По-задълбоченото взаимодействие между дизайна на **невронната мрежа** и **сходимостта** в обучението с подкрепление — особено в контекста на **агенти за игри** — е отложено за Стъпка 5.

Прочетете повече: Schulman, J. и др. (2017). „Алгоритми за оптимизация на проксималната стратегия.“

arXiv: <https://arxiv.org/abs/1707.06347>

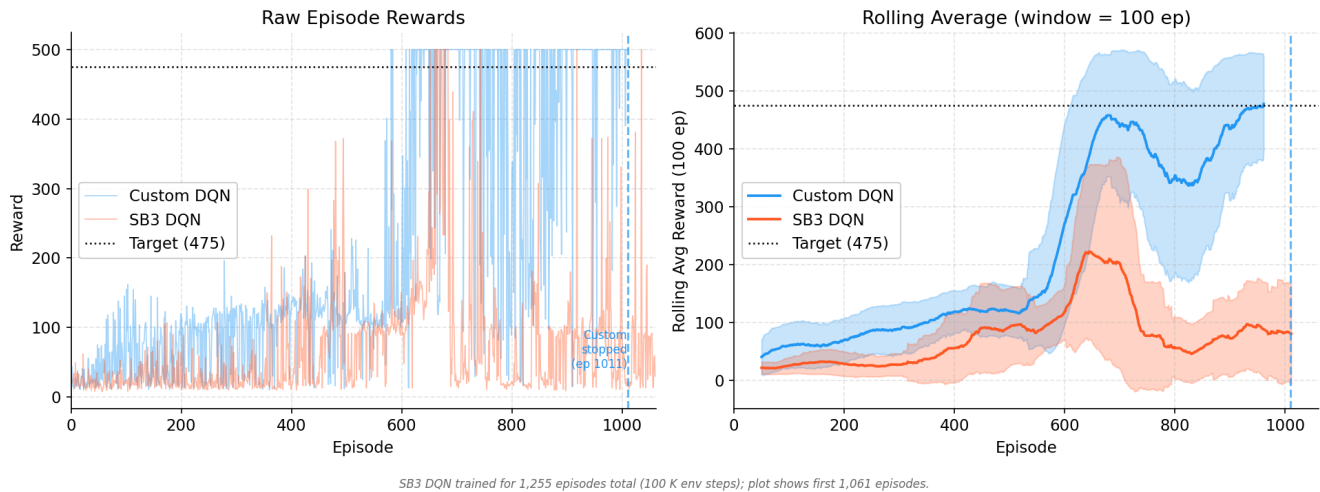
1.9 Практическо валидиране — собствени имплементации спрямо stable-Baselines3

За да проверим дали нашите имплементации от нулата функционират коректно, ги сравнихме със stable-Baselines3 (SB3) — широко използвана и добре тествана RL библиотека. На нашите имплементации и на SB3 бяха зададени еднакви хиперпараметри и еквивалентни бюджети за обучение.

1.9.1 DQN върху cartPole-v1

Нашият собствен DQN решава CartPole около епизод ~1011, като достига 100-епизодна плъзгаща средна от 477,5 (над целта от 475). DQN на SB3, с предоставени 750K стъпки на средата (17 176 епизода), не успява да се сближи — неговата плъзгаща средна остава около 30 и никога не достига целта.

Разликата се дължи на три различия на ниво изпълнение:



Фигура 1: DQN comparison — Custom vs SB3

- **График на изследване.** Нашият епсилон намалява с всеки епизод ($\times 0,995$), като така концентрира изследването там, където то е най-необходимо. SB3 намалява епсилон линейно спрямо броя стъпки, което в среди с кратки епизоди като cartPole води до загуба на много епизоди в почти случайна мода.
- **Синхронизация на целевата мрежа.** Нашият код синхронизира на всеки 5 епизода — адаптивен план, който осигурява по-бърза обратна връзка в ранните етапи на обучението. SB3 използва фиксиран интервал от 1000 стъпки.
- **Ранно спиране.** Нашият агент запазва най-добрия модел и прекратява обучението при достигане на пикова ефективност, като по този начин избягва катастрофално забравяне. SB3 изпълнява целия предвиден бюджет и стратегията му започва да се колебае.

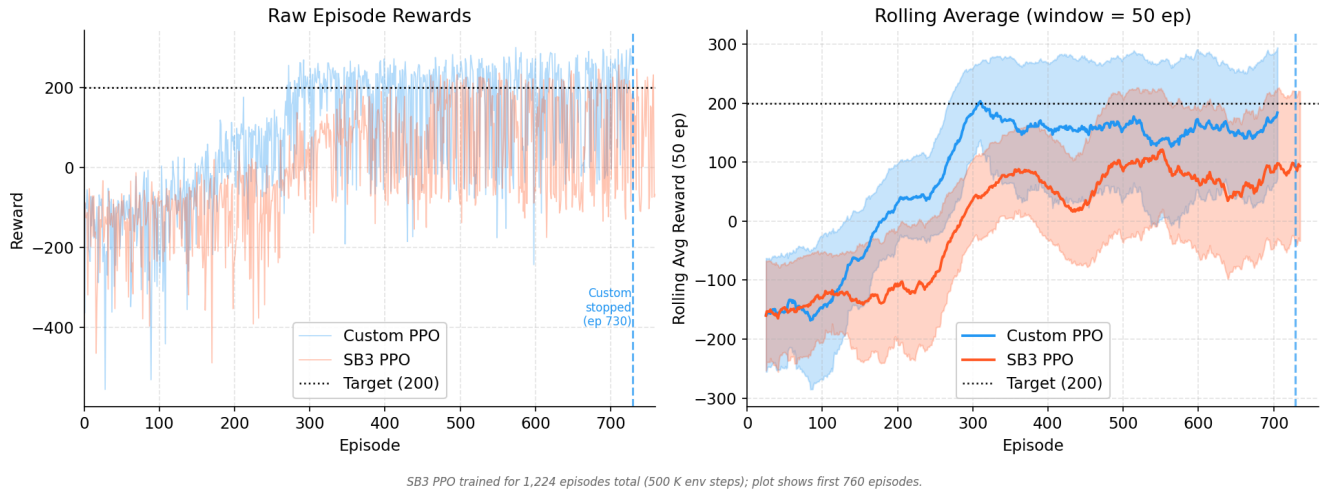
Това са истински алгоритмични решения, а не несправедливост при хиперпараметрите. Настройките по подразбиране на SB3 са проектирани за стабилна работа в разнообразни и дългосрочни области на обучение като Atari Games — а не в класически среди за управление с кратки епизоди като cartPole; нашите избори, съобразени със спецификата на задачата, просто дават по-добри резултати при cartPole.

1.9.2 PPO върху lunarLander-v3

Нашият персонализиран PPO преминава целта от 200 около епизод 543 (с бюджет от 264K стъпки). PPO на SB3 със същия бюджет от 500K стъпки достига връх с най-добра плъзгаща се средна стойност за 100 епизода от 131,2 — все още се покачва, но не е сходим.

Разликата тук е по-малка, отколкото при DQN. И двата използват идентични хиперпараметри на PPO. Оставащата разлика вероятно се дължи на ортогоналната инициализация на теглата и вградената нормализация на наблюдението в SB3, които подпомагат по-дългите серии от изпълнения, но могат да забавят ранната сходимост. При предоставяне на по-голям обучителен бюджет (1–2M стъпки), PPO на SB3 вероятно би достигнала целта.

PPO on LunarLander-v3 — Custom vs SB3



Фигура 2: PPO comparison — Custom vs SB3

1.9.3 Извод

Чиста реализация от нулата обхваща около 300–500 реда код, докато тази на SB3 достига приблизително 10 000. Компромисът е по-малко функции срещу пълна прозрачност и лесна възможност за отстраняване на грешки. За изследователски цели — особено при планираната работа върху експлоатацията на противника в стъпки 7–8 — персонализираните реализации дават възможност за прецизни модификации (например добавяне на наблюдения на състоянието на убежденията), които биха изисквали значително подклаждане в SB3. Именно тази практическа гъвкавост беше причината да изградим решение от нулата.